

Project Workflow: Week 2

Phase: Requirements Analysis & System Design

1. Objective

The goal of Week 2 was to gather all functional requirements for the game and translate them into a clear system design. This included:

1. Identifying and documenting what the game needs to do — the road system, obstacle cars, scoring, and gas pickups.
2. Designing the object-oriented class structure to organise the code cleanly before any implementation begins.
3. Creating basic wireframe descriptions for the game window layout and HUD overlay.

2. Detailed Task Breakdown

Gathering Functional Requirements

Listed all the features the game must have to work correctly. This covered the 3-lane scrolling road with curve behaviour, the player car's lane-switching and speed-boost controls (arrow keys and space bar), randomised obstacle cars that come from both directions, gas canister pickups that restore speed, a live HUD showing score, speed, and distance, and a win condition triggered when the player travels 5,000 units.

Object-Oriented Class Architecture Design

Designed the full class structure for the game before writing any code. The plan covers:

1. CarRacingGame (JFrame) — the main entry point that creates the window and starts the game.
2. GamePanel (JPanel) — handles the core game loop, rendering pipeline, keyboard input, and game state (running, game over, won).
3. Road — manages the 3-lane scrolling road, lane positions, and curve physics.

4. Car (abstract) — the base class holding shared properties like position, size, colour, and collision logic, extended by PlayerCar and EnemyCar.
5. PlayerCar — extends Car; handles lane-switching, speed boosting, and gas collection.
6. EnemyCar — extends Car; spawns with randomised colours, speeds, and directions (towards or away from the player).
7. Environment — manages all background scenery objects: Tree, Bush, Mountain, Lake, Park, Bird, and Cloud.
8. GasItem — represents a collectible pickup that gives the player a speed boost and adds to the score.

Defining Class Interactions

Mapped out how the classes communicate during each game update cycle. GamePanel calls update on the Road, Environment, PlayerCar, each EnemyCar, and each GasItem every frame. It checks for collisions between PlayerCar and EnemyCar (ending the game) and between PlayerCar and GasItem (collecting the pickup). Scoring is updated when an EnemyCar is successfully passed, and the win condition is checked against the total distance travelled.

Basic Wireframes for the Game Window and HUD Overlay

Planned the basic layout of the game window and the HUD overlay in written form before any coding begins.

Game Window Layout: The window is 900x700 pixels. The road occupies the centre of the screen and is divided into 3 lanes. The left and right sides of the road show the grass and scenery area where trees, bushes, lakes, and parks appear. The sky and background mountains are visible in the upper portion of the screen. Enemy cars travel along the road towards and away from the player. Gas canister pickups appear in any of the 3 lanes and scroll down towards the player.

HUD Overlay Layout: A semi-transparent panel sits in the top-left corner of the screen showing three pieces of live information — the current score, the current speed in km/h, and the distance travelled out of the 5,000-unit goal. When the player activates the speed boost, a boost indicator appears at the top centre of the screen. On game over or win, a full-screen overlay appears showing the final score and a prompt to press R to restart.

3. Deliverables

1. Requirements Document: A written list of all functional features the game must include.
2. Class Architecture Plan: A written description of all classes, their responsibilities, and how they relate to each other.
3. Interaction Overview: A written summary of how the classes communicate and update each game frame.
4. Wireframe Description: A written layout plan for the game window and HUD overlay.

4. Tools & Resources

Development Environment

1. Java JDK 8+
2. IntelliJ IDEA / VS Code

Reference Materials

Java Swing and AWT 2D graphics documentation, object-oriented design principles, and javax.swing.Timer-based game loop references.